

Homework 2

Часть 1: Выбор Сценария

Пожалуйста, выберите **ОДИН** из следующих бизнес-сценариев для проектирования вашей базы данных:

1. **Управление строительными проектами:** Управление проектами, сотрудниками, клиентами и задачами проекта.
2. **Система бронирования в ресторане:** Управление бронированиями клиентов, столиками, персоналом и пунктами меню.
3. **Система высшего образования:** Студенты, предметы, преподаватели, расписание и т. д.
4. **Сервис потоковой передачи музыки:** Управление артистами, альбомами, песнями, пользователями и пользовательскими плейлистами.
5. **Управление арендной недвижимостью:** Отслеживание арендной недвижимости, арендаторов, договоров аренды и запросов на обслуживание.
6. **Онлайн-магазин электроники:** Управление категориями товаров, товарами, покупателями и оформленными заказами.
7. **Управление фитнес-клубом:** Управление клиентами, тренерами, групповыми занятиями и записью клиентов на тренировки.
8. **Сервис аренды электросамокатов:** Отслеживание пользователей, самокатов, парковочных зон и совершенных поездок.
9. **Ветеринарная клиника:** Управление владельцами питомцев, самими питомцами, врачами-ветеринарами и приемами.
10. **Продажа билетов на мероприятия:** Управление площадками, мероприятиями, посетителями и купленными билетами.

Часть 2: Проектирование Базы Данных и Документация

На основе выбранного вами сценария вы должны:

1. **Определить сущности и атрибуты:** Определите основные сущности, относящиеся к выбранному вами сценарию, и перечислите их ключевые атрибуты.
2. **Спроектировать таблицы:** Для каждой определенной сущности, которая станет таблицей, предоставьте следующие сведения:
 - **Имя таблицы**
 - **Описание**
 - **Атрибуты:** Перечислите все атрибуты. Для каждого атрибута укажите:
 - Тип данных
 - Первичный ключ (PK)

- Внешний ключ (FK)
 - Ограничения
3. **Нормализация (3NF):** Убедитесь, что все ваши таблицы находятся в третьей нормальной форме (3NF).
 4. **Связи:** Четко опишите связи между вашими таблицами, указав их мощность (например, «один-к-одному», «один-ко-многим», «многие-ко-многим») и участвующие сущности.

Контрольный список требований:

- Минимум 4 таблицы (сущности).
- Все таблицы включают первичный ключ (PK).
- Соответствующие типы данных для всех атрибутов.
- Минимум одно ограничение на таблицу (помимо PK).
- Реализована как минимум одна связь «многие-ко-многим».
- Все связи четко определены с помощью внешних ключей (FK).

Часть 3: ER-диаграмма

Предоставьте физическую ER-диаграмму, представляющую выбранный вами сценарий:

- Покажите таблицы с конкретными именами столбцов, типами данных, подходящими для конкретной системы баз данных SQL (PostgreSQL).
- Включите ограничения первичного ключа, внешнего ключа, NOT NULL и UNIQUE с использованием синтаксиса SQL.

Вы можете использовать любой инструмент для создания ER-диаграмм (например, [Draw.io](https://draw.io), pgModeler, Lucidchart, MySQL Workbench, Miro, ER/Studio, Oracle SQL Developer Data Modeler или даже нарисованный от руки, если он разборчивый).

Отправка

Ваша работа должна включать:

- **Выбранный сценарий:** Четко укажите, какой из пяти сценариев вы выбрали.
- **Подробные проекты таблиц:** Для каждой таблицы укажите имя, описание, атрибуты, типы данных, обозначение PK/FK и ограничения, как указано в части 2.
- **Объяснение связей:** Опишите каждую связь и ее мощность.
- **Физические ER-диаграммы**

Пример отчёта

Part 1: Выбор Сценария

Для данной работы выбран сценарий: **Система Управления Библиотекой**. Эта система будет управлять книгами, читателями, авторами и выдачей книг.

Part 2: Проектирование Базы Данных и Документация

Идентификация Сущностей и Атрибутов:

1. **Книги (Books):**
2. **Авторы (Authors):**
3. **Читатели (Readers):**
4. **Выдача Книг (BookLoans):** (Для отслеживания, кто какую книгу взял)

Проектирование Таблиц:

1. Table Name: Authors

- **Description:** Хранит информацию об авторах.
- **Attributes:**
 - *AuthorID*: INTEGER, PK, NOT NULL, UNIQUE
 - *FirstName*: VARCHAR(100), NOT NULL
 - *LastName*: VARCHAR(100), NOT NULL
- **Constraints:**
 - *PK_Authors*: PRIMARY KEY (AuthorID)
 - *UQ_AuthorFullName*: UNIQUE (FirstName, LastName)

2. Table Name: Books

- **Description:** Содержит информацию о книгах.
- **Attributes:**
 - *BookID*: INTEGER, PK, NOT NULL, UNIQUE
 - *Title*: VARCHAR(255), NOT NULL
 - *PublicationYear*: INTEGER
 - *AuthorID*: INTEGER, FK (REFERENCES Authors), NOT NULL
- **Constraints:**
 - *PK_Books*: PRIMARY KEY (BookID)
 - *CHK_PublicationYear*: CHECK (PublicationYear >= 1000 AND PublicationYear <= 2100)
 - *FK_Books_Authors*: FOREIGN KEY (AuthorID) REFERENCES Authors(AuthorID)

3. Table Name: Readers

- **Description:** Хранит данные о читателях библиотеки.
- **Attributes:**

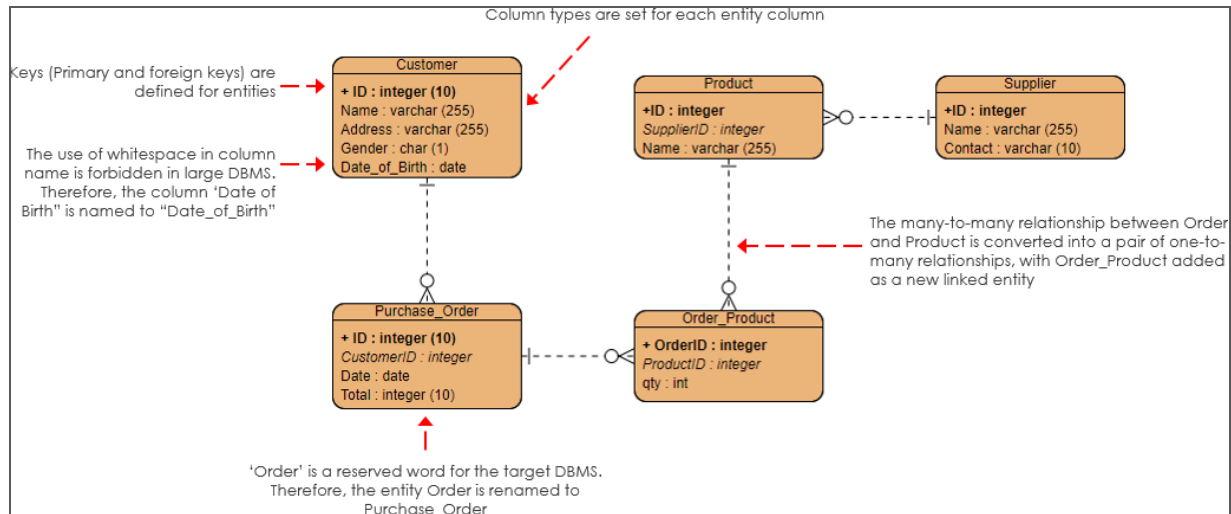
- **ReaderID:** INTEGER, PK, NOT NULL, UNIQUE
 - **FirstName:** VARCHAR(100), NOT NULL
 - **LastName:** VARCHAR(100), NOT NULL
 - **Email:** VARCHAR(255), UNIQUE
 - **Constraints:**
 - PK_Readers: PRIMARY KEY (ReaderID)
 - UQ_Email: UNIQUE (Email)
4. **Table Name: BookLoans**
- **Description:** Таблица для реализации связи многие-ко-многим. Записывает информацию о выдаче книг читателям.
 - **Attributes:**
 - *LoanID*: INTEGER, PK, NOT NULL, UNIQUE
 - *BookID*: INTEGER, FK (REFERENCES Books), NOT NULL
 - *ReaderID*: INTEGER, FK (REFERENCES Readers), NOT NULL
 - *LoanDate*: DATE, NOT NULL, DEFAULT CURRENT_DATE
 - *ReturnDate*: DATE
 - **Constraints:**
 - PK_BookLoans: PRIMARY KEY (LoanID)
 - FK_BookLoans_Books: FOREIGN KEY (BookID) REFERENCES Books(BookID)
 - FK_BookLoans_Readers: FOREIGN KEY (ReaderID) REFERENCES Readers(ReaderID)
 - CHK_Dates: CHECK (ReturnDate IS NULL OR ReturnDate >= LoanDate)

Взаимосвязи:

- **Authors и Books (Один-ко-Многим):** Одному автору может принадлежать множество книг, но каждая книга в этом упрощенном варианте имеет одного основного автора.
 - *Books.AuthorID* является внешним ключом, ссылающимся на *Authors.AuthorID*.
- **Books и BookLoans (Один-ко-Многим):** Одна книга может быть выдана множество раз, но каждая запись о выдаче относится к одной конкретной книге.
 - *BookLoans.BookID* является внешним ключом, ссылающимся на *Books.BookID*.
- **Readers и BookLoans (Один-ко-Многим):** Один читатель может взять в долг множество книг (много записей о выдаче), но каждая запись о выдаче относится к одному читателю.
 - *BookLoans.ReaderID* является внешним ключом, ссылающимся на *Readers.ReaderID*.

Part 3: ER-Диаграмма

Пример ER диаграммы. От вас ожидается похожая диаграмма, только с сущностями выбранного вами сценария



Требования к сдаче домашнего задания

Результатом домашнего задания является скриншот ER диаграммы, docx файл с описанием спроектированной базы данных и описанием какие цели и задачи решает ваш сценарий.

Опционально: представьте что вы пишете документацию к вашему проекту, где вы описываете как бы “дорожную карту” по проекту. Возьмите ваш отчёт (docx файл) за референс и приведите его к **markdown** формату (.md). При приведении к .md формату можно использовать AI. Конкретных требований к оформлению данного документа нету.

Для удобства проверки домашних заданий, соблюдайте следующие правила работы с репозиторием Git:

- Каждое домашнее задание необходимо коммитить в отдельную ветку, создаваемые от ветки main.
- Имя ветки должно строго соответствовать формату <lesson_name>_<task_number> (db_normalization_2, python_loops_7)
- Для сдачи задания необходимо создать Pull Request в main-ветку.
- В Pull Request обязательно должен быть добавлен ментор и руководитель лаборатории как reviewers, чтобы он могли проверить и оставить комментарии.
- Не мержите PR, дождитесь проверки от reviewers.

Пошаговая инструкция как правильно отправить задание

1. Переключиться на main и обновить её:
git checkout main - переключение на main
git pull origin main - обновить проект локально на вашем компьютере
2. Создать ветку для домашнего задания:
git checkout -b db_normalization_2
3. Добавить нужные файлы и закоммитить изменения:
git add . (точка означает добавить в коммит все новые файлы/изменения)
git commit -m "Add homework 2" (обязательно указывайте осмысленные комментарии к коммиту)
4. Отправить ветку на сервер:
git push origin db_normalization_2
5. Создать **Pull Request**:
 - Убедитесь, что целевая ветка — main.
 - Название PR должно быть понятным, например: Homework 2: DB, Normalization
 - Обязательно добавьте ментора и руководителя лаборатории в reviewers.
6. Ожидайте проверки и не мержите PR самостоятельно.